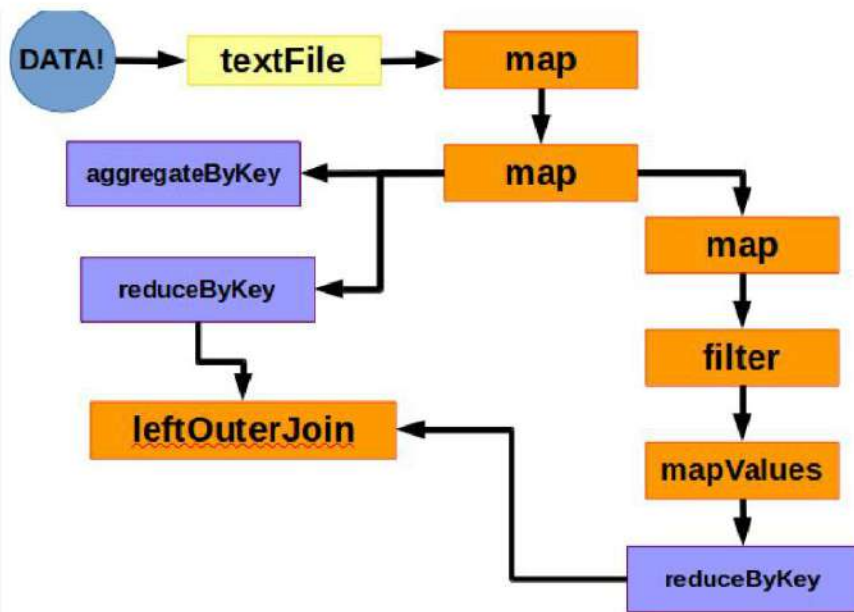


PYSPARK INTERVIEW CHEATSHEET



PySpark Cheat Sheet

READING DATA

```
spark.read.csv("file.csv")
spark.read.json("file.json")
spark.read.parquet("file.parquet")
```

Aggregations

```
df.groupBy("col")
df.agg("col": "mean")
df.groupBy("col").count()
```

TRANSFORMATIONS

```
map() = map()
flatMap(f) = flatmap()
filter(f) = filter()
union(rdd) = union(rdd)
```

To add a new column;

```
df.withColumn("new_col",
df.withColumn(y, squarefx))
```

MACHINE LEARNING

```
from pyspark.ml.feature
import VectorAssembler
```

MACHINE LEARNING

```
from pyspark in Pyshmode
```

TRANSFORMATIONS

```
map(f)
flatMap(f)
filter(f)
union(rdd)
reduceByKey(f)
```

ACTIONS

```
collect()
count()
take(n)
saveAsTextFile(path)
```

DATES

```
current.date()
datediff(df.date, df.date)
df.withColumn("date",
f.to.date(f.col))
```

UDFs: User-defined Functions

```
define method &
using a custom
User-defined Function
```

MACHINE LEARNING

```
from pyspark.ml
in Pysmobule VectorAss
```

0) ABSOLUTE BASICS

```
from pyspark.sql import SparkSession
spark = (SparkSession.builder
        .appName("app")
        .config("spark.sql.shuffle.partitions", "200")
        .getOrCreate())

sc = spark.sparkContext # low-level RDD ctx
```

- **RDD vs DataFrame:** RDD = untyped, functional API; DataFrame = typed schema, Catalyst optimizer, Tungsten (faster). Prefer DataFrames.
- **Lazy + DAG:** Transformations build a DAG; an **action** triggers execution.
- **Narrow vs Wide:** narrow (map, filter) = no shuffle; wide (groupBy, join, distinct) = shuffle.

1) TRANSFORMATIONS & ACTIONS (MOST-ASKED)

Transformations: select, withColumn, filter, drop, join, groupBy, agg, orderBy, repartition, coalesce, distinct, unionByName, explode, withWatermark (streaming).

Actions: count, collect, take, show, head, foreach, write.

```
from pyspark.sql import functions as F

df = spark.read.json("s3://bucket/events/")
df2 = (df
      .filter(F.col("status") == "OK")
      .withColumn("dt", F.to_date("ts"))
      .groupBy("dt").agg(F.count("*").alias("n")))
df2.orderBy("dt").show()
```

2) SCHEMA & I/O

```
from pyspark.sql.types import *
schema = StructType([
    StructField("id", LongType()),
    StructField("ts", TimestampType()),
    StructField("attrs", MapType(StringType(), StringType()))
])
df = spark.read.schema(schema).json("/data/in.json") # avoid inferSchema on big data
df.write.mode("overwrite").partitionBy("dt").parquet("/data/out")
```

- Prefer **Parquet/ORC** (columnar, predicate pushdown).
- Use **partition pruning**: partition by low-cardinality, frequently filtered columns (e.g., date).

3) JOINS (AND PERFORMANCE)

```
# Types: inner, left, right, full, left_semi, left_anti, cross
small = spark.read.parquet("/ref/dim_country")
fact = spark.read.parquet("/fact/sales")
```

```
# Broadcast hint (when small << big)
joined = fact.join(F.broadcast(small), "country_id", "left")
```

- **Skew:** use salting (add random bucket to key), or `skewHint`, or broadcast the smaller side.
- **Join reordering:** Catalyst + AQE can auto-optimize if stats exist.

4) WINDOW FUNCTIONS

```
from pyspark.sql.window import Window
w = Window.partitionBy("user_id").orderBy(F.col("ts").asc())
```

```
out = (df
    .withColumn("rn", F.row_number().over(w))
    .withColumn("prev_val", F.lag("value").over(w))
    .withColumn("sum7", F.sum("value").over(
        Window.partitionBy("user_id")
            .orderBy("ts")
            .rowsBetween(-6, 0)))
    )
```

Common: `row_number`, `rank`, `dense_rank`, `lag/lead`, `running sums/avgs`, `sessionization via gaps` (in Streaming use `session_window`).

5) NULLS, DATES, COMPLEX TYPES

```
df = (df
    .fillna({"country": "UNKNOWN"})
    .withColumn("yyyy_mm", F.date_format("ts", "yyyy-MM"))
    .withColumn("age_bucket", F.when(F.col("age") < 18, "teen")
        .when(F.col("age") < 30, "20s")
        .otherwise("30+"))
    .withColumn("city", F.get_json_object("json_blob", "$.address.city"))
    .withColumn("tag", F.element_at("tags", 1)) # arrays are 1-based in SQL
    .withColumn("kv", F.map_from_entries("pairs")))
```

6) CACHING, PARTITIONS & SHUFFLE

```
df_cached = df.repartition(200, "dt").persist() # MEMORY_AND_DISK by default
df_cached.count()
df_cached.unpersist()
```

- `repartition(n, cols)` = shuffle; `coalesce(n)` = no shuffle (reduce only).

- Set `spark.sql.shuffle.partitions` sensibly (not the default 200 for small jobs; increase for large shuffles).
- Watch stage boundaries: wide ops cause new stages.

7) UDFS (AND PANDAS UDFS)

- Prefer **built-in functions**. UDFs break optimization & pushdown.
- If needed, use **Pandas UDFs (vectorized)** for speed.

```
# Scalar Pandas UDF
@F.pandas_udf("double")
def zscore(col: pd.Series) -> pd.Series:
    return (col - col.mean()) / col.std()

df2 = df.withColumn("z", zscore("value"))

# Grouped map Pandas UDF
schema = "user_id long, ts timestamp, rank int"
@F.pandas_udf(schema, F.PandasUDFType.GROUPED_MAP)
def rank_in_group(pdf):
    pdf = pdf.sort_values("ts")
    pdf["rank"] = range(1, len(pdf)+1)
    return pdf

df.groupBy("user_id").apply(rank_in_group)
```

8) ADAPTIVE QUERY EXECUTION (AQE) & OPTIMIZER

- **Catalyst**: logical plan → optimized plan → physical plan.
- **Tungsten**: whole-stage codegen, off-heap memory, binary format.
- **AQE (Spark 3+)**: coalesces shuffle partitions, switches join strategies at runtime, and handles skew.

```
spark.conf.set("spark.sql.adaptive.enabled", "true") #
usually default true
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 64*1024*1024) #
64MB
```

9) SKEW HANDLING PATTERNS

- **Broadcast** small side (`broadcast()` hint).
- **Salting**: add random bucket to skewed key; join on (key, salt).
- **AQE Skew Join**: enable configs above.
- **Pre-aggregate** before join (`groupBy` to shrink rows).

10) FILE LAYOUT & SMALL FILES

- Write fewer, larger files: `df.coalesce(N).write...`
- To compact existing small files: `read` → `repartition/coalesce` → `overwrite`.
- For tables, use **OPTIMIZE**/compaction utilities if Delta/Iceberg.

11) DELTA/ICEBERG (IF ASKED)

- **Delta Lake:** ACID, time travel, MERGE/UPSERT, vacuum, Z-order.

```
(spark.read.format("delta").load("/tables/sales")
  .createOrReplaceTempView("sales"))
spark.sql("""
MERGE INTO sales s USING updates u
ON s.id = u.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")
```

- **Iceberg:** table format with hidden partitioning, snapshots, branching; SQL: `CALL system.rewrite_data_files` for compaction.

12) STRUCTURED STREAMING ESSENTIALS

```
stream = (spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", "broker:9092")
  .option("subscribe", "topic").load())

parsed = stream.select(F.col("value").cast("string").alias("raw"))
agg = (parsed
  .withColumn("ts", F.current_timestamp())
  .withWatermark("ts", "10 minutes")
  .groupBy(F.window("ts", "5 minutes")).count())

query = (agg.writeStream.outputMode("update")
  .format("console")
  .option("checkpointLocation", "/chk/app1")
  .start())
# awaitTermination() in real apps
```

- **Exactly-once** with idempotent sinks or Delta; always set **checkpointLocation**.
- Watermark to bound state; **trigger** options (`processingTime`, `availableNow`).

13) CLUSTER & EXECUTION MODEL

- **Driver** plans DAG; **executors** run tasks; **cluster manager** (YARN, K8s, Standalone).
- **Stage** = set of tasks between shuffles; **task** = unit of parallelism per partition.
- Key resource knobs: executor cores, memory, instances; `spark.task.maxFailures`, `spark.memory.fraction`.

14) ACCUMULATORS & BROADCAST VARIABLES

```
acc = sc.longAccumulator("bad_rows")
def mark_bad(row):
    if row.bad: acc.add(1)
df.foreach(lambda r: mark_bad(r))
```

```
bc_map = sc.broadcast({"IN":"India", "US":"United States"})
```

- Accumulators are write-only in executors (read on driver).
- Broadcast to avoid shipping large lookup objects per task.

15) COMMON "GOTCHAS"

- **Actions in a loop** → multiple DAG runs; cache if reused.
- **collect() on big DF** → OOM on driver; use `limit`, `take`.
- **orderBy().show()** without `limit` can be expensive.
- **Schema drift**: set `mergeSchema` if needed (Parquet/Delta).
- **Time zones**: set `spark.sql.session.timeZone`.

16) HANDY ONE-LINERS

```
# Read multiple paths & union by name
df = spark.read.parquet(*["/p/y=2025/m=10", "/p/y=2025/m=11"])
df_all = df1.unionByName(df2, allowMissingColumns=True)

# Top N per group
w = Window.partitionBy("grp").orderBy(F.col("metric").desc())
topN = df.withColumn("rn", F.row_number().over(w)).filter("rn <= 3")

# Explode and re-aggregate
tags = (df.select("id", F.explode_outer("tags").alias("tag"))
        .groupBy("tag").count())

# Write partitioned & bucketed (Hive/Delta tables only)
(df.write
  .mode("overwrite")
  .bucketBy(32, "user_id")
  .sortBy("user_id")
  .saveAsTable("db.fact_users_bucketed"))
```

17) PERFORMANCE CHECKLIST (RAPID-FIRE)

- Use DataFrames, built-ins, and **avoid Python UDFs** unless vectorized.
- **Filter early, select needed columns only.**
- **Broadcast small dimension tables**; verify with `explain()` and `spark.sql.adaptive.*`.
- **Right partition count**: repartition before wide agg/join; **coalesce** before writing.
- **Cache only if reused ≥ 2 times; unpersist.**
- **Columnar formats; predicate pushdown.**
- **Handle skew** (broadcast, salting, AQE).
- **Compact small files.**
- Profile with `df.explain("formatted")` and **Spark UI** (look at stages, spills, skew).

18) MINI Q-A SNIPPETS (SOUND CONFIDENT)

- **How does Spark optimize DataFrames?**

- CATALYST BUILDS A LOGICAL PLAN, APPLIES RULE-BASED & COST-BASED OPTIMIZATIONS (PREDICATE PUSHDOWN, CONSTANT FOLDING, JOIN REORDERING), THEN CHOOSES A PHYSICAL PLAN. TUNGSTEN USES OFF-HEAP MEMORY & CODEGEN FOR CPU EFFICIENCY. AQE REFINES PLANS AT RUNTIME.
- **Broadcast vs Shuffle Hash Join?**
BROADCAST SHIPS THE SMALL TABLE TO ALL EXECUTORS (FAST, AVOIDS SHUFFLE). SHUFFLE/HASH JOINS REQUIRE SHUFFLING BOTH SIDES BY KEY (EXPENSIVE BUT SCALABLE).
- **When to use cache vs checkpoint?**
CACHE FOR PERFORMANCE (CAN BE EVICTED). CHECKPOINT TO TRUNCATE LINEAGE FOR LONG DAGS/FAULT TOLERANCE; WRITES TO RELIABLE STORAGE AND CLEARS LINEAGE.

19) QUICK CONFIGS YOU'LL QUOTE

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 100*1024*1024) #
100MB
spark.conf.set("spark.sql.files.maxPartitionBytes", 128*1024*1024)
spark.conf.set("spark.sql.shuffle.partitions", 400) # tune to data size
```